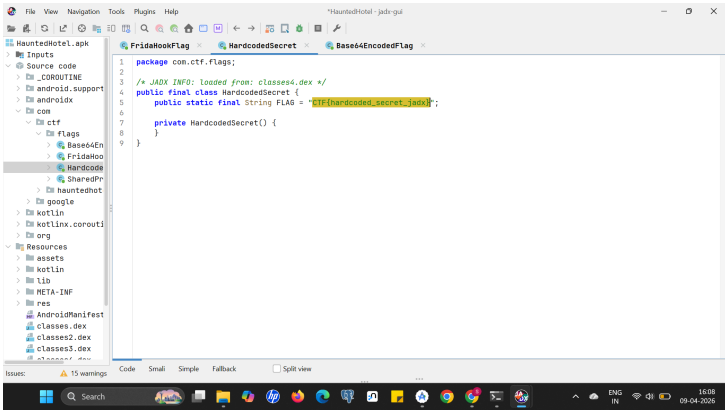
Haunted hotel
Mobile Security Lab Package
Detailed CTF documentation

- Hamanth M(22pc13)
- Sornamalya(22pc34)

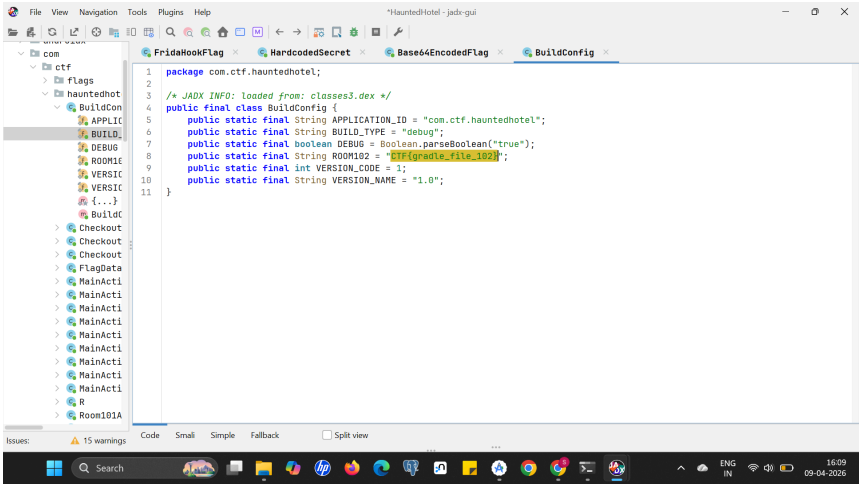
FLAG 1 — Hardcoded Secret (JADX)

What to find:
Class: com.ctf.flags.HardcodedSecret
Field: FLAG
Value: CTF{hardcoded_secret_jadx}
How to get it:
1) Extract or build the APK (assembleDebug creates app-debug.apk).
2) Open the APK in JADX-GUI.
3) Navigate to com.ctf.flags.HardcodedSecret.
4) Read public static final String FLAG.



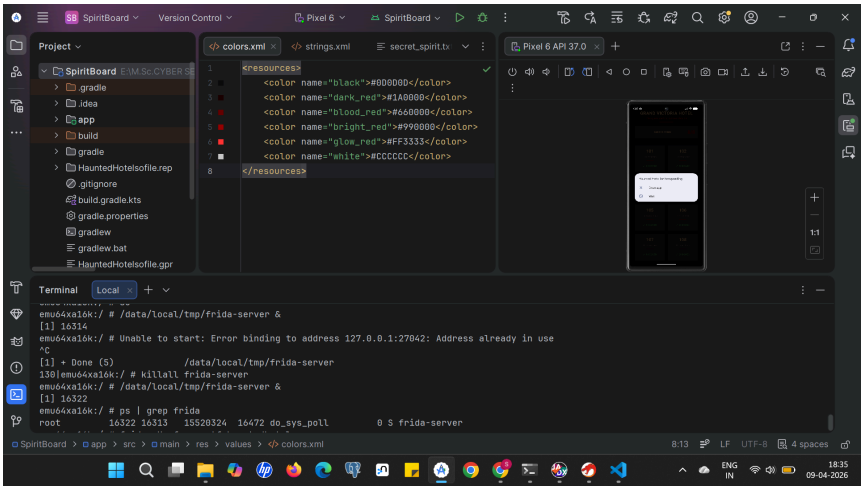
FLAG 2 — Build Config file (JADX)

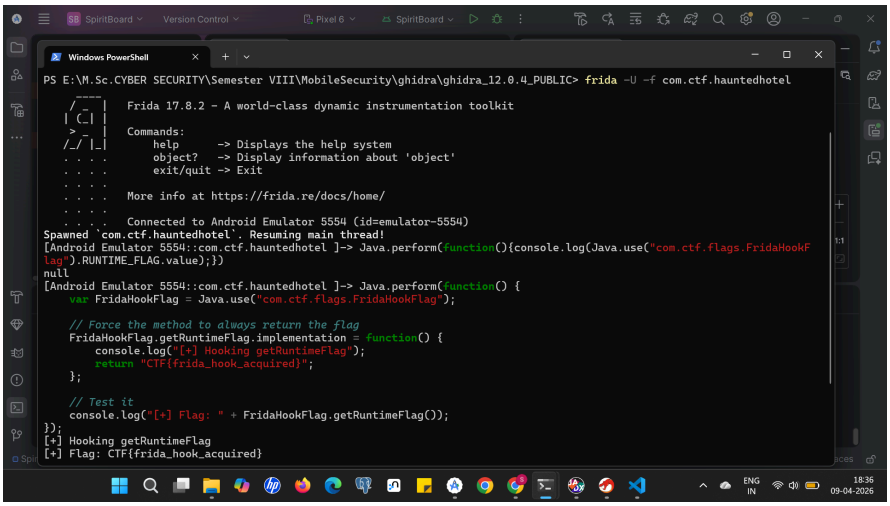
1.Decompile the APK using jadx-gui.
2.Navigate to com.ctf.hauntedhotel.BuildConfig.java which is created at runtime.
3.Read the flag directly: "CTF{gradle_config_102}".
4.Enter the flag in Room 102 to free the ghost.



FLAG 3 — Frida Hook (Runtime Flag)

What to find:
Class: com.ctf.flags.FridaHookFlag
Method: getRuntimeFlag()
Behavior: returns "CTF{frida_hook_acquired}" when a debugger is attached or “Frida-likely” classes are present; otherwise returns "NO_FLAG".
How to get it:
Option A: Attach a debugger
1) Start the app on a device in debug.
2) Attach Android Studio debugger.
3) Use a method invocation (JDWP Evaluate Expression) on FridaHookFlag.getRuntimeFlag() if available in your setup, or use Frida below.
Option B: Use Frida directly
1) Start the app.
2) Frida script to call the static method:
Java.use("com.ctf.flags.FridaHookFlag").getRuntimeFlag() and print the result.
Since Frida is attached, the method should return the flag.
Why this works:
The class explicitly checks Debug.isDebuggerConnected() and attempts to detect Frida; those conditions flip the method’s return to the flag.





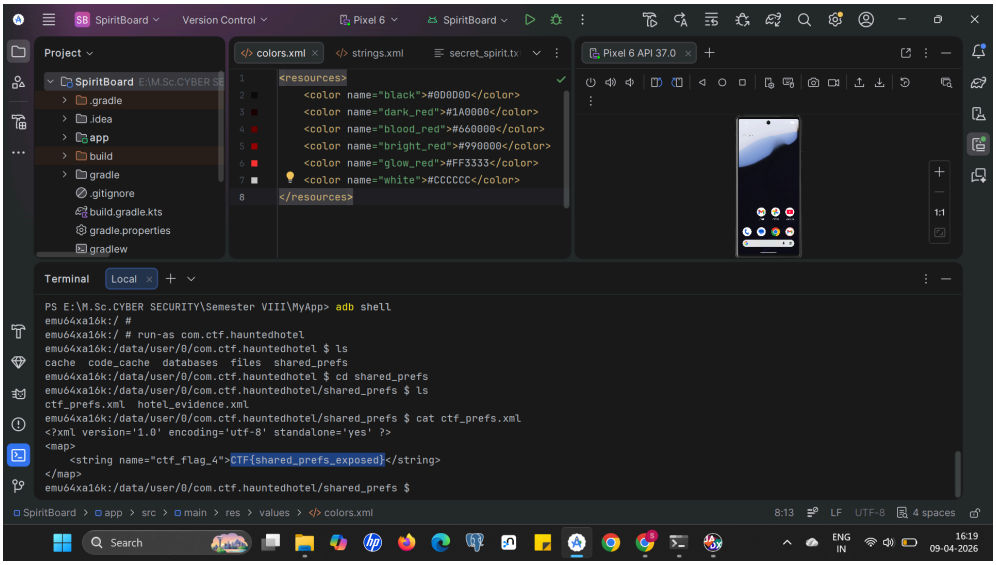
FLAG 4 — SharedPreferences Leak

What to find:
Class: com.ctf.flags.SharedPreferencesLeak
Prefs:
Name: ctf_prefs
Key: ctf_flag_4
Value (if written): FLAG{shared_prefs_exposed}
How to get it:
Step 1: Ensure the flag is written (CTF hint)
Any component can call SharedPreferencesLeak.writeFlag(context); if not already invoked by app flow, you can:
Use Frida to call SharedPreferencesLeak.writeFlag(currentActivity) at runtime.
Or call it via another hook once you get a Context.
Step 2: Read from SharedPreferences
On a debug build, use run-as to read the file:
Package: com.ctf.hauntedhotel

Path: /data/data/com.ctf.hauntedhotel/shared_prefs/ctf_prefs.xml
Commands:

```
adb shell run-as com.ctf.hauntedhotel
cat /data/data/com.ctf.hauntedhotel/shared_prefs/ctf_prefs.xml
```

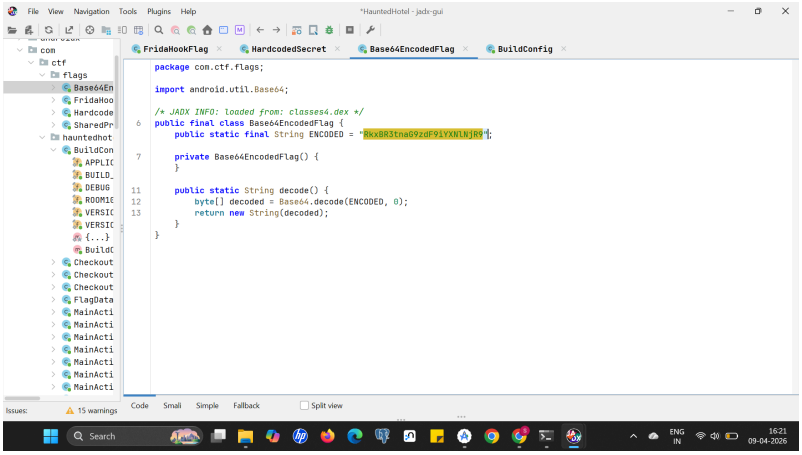
Look for <string name="ctf_flag_4">FLAG{shared_prefs_exposed}</string>.
Why this works:
Debuggable apps allow adb shell run-as to read app-internal files; SharedPreferences are plaintext XML.

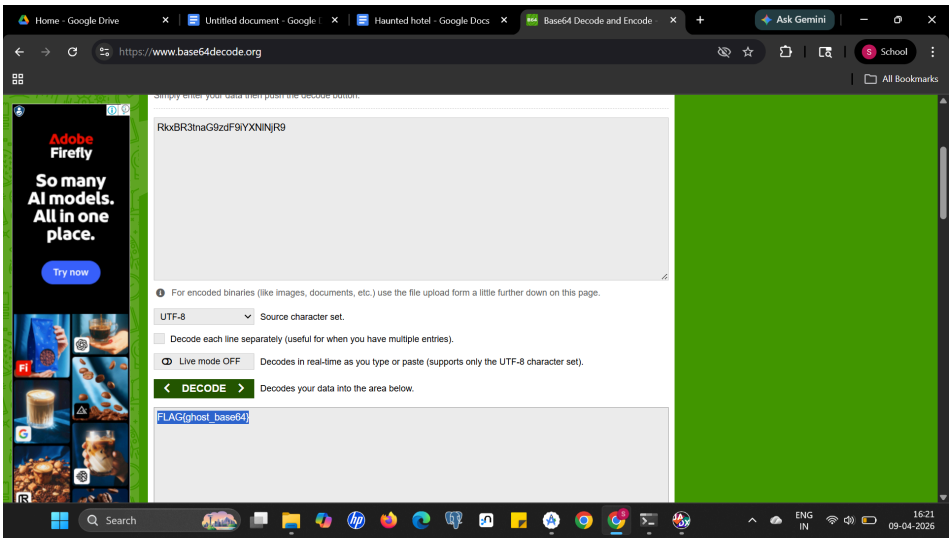


FLAG 5 — Base64 Encoded String

What to find:
Class: com.ctf.flags.Base64EncodedFlag
Field: ENCODED = "RkxBR3tnaG9zdF9iYXNlNjR9"
Method: decode() returns the plaintext flag.
How to get it:
Static:
1) Open APK in JADX.
2) Navigate to Base64EncodedFlag.
3) Decode "RkxBR3tnaG9zdF9iYXNlNjR9" using any Base64 tool to get FLAG{ghost_base64}.
Dynamic:
1) Use Frida to call Base64EncodedFlag.decode() and print the result.
Why this works:
Base64 provides no secrecy; it's trivially decoded once located.
Notes for players
Package name: com.ctf.hauntedhotel.
APK location (debug): app/build/outputs/apk/debug/app-debug.apk.

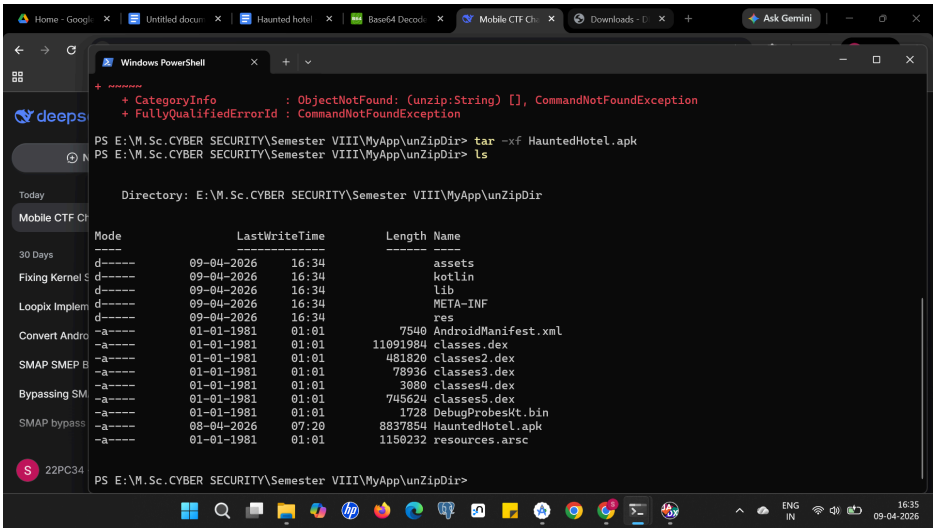
Common tools:
Static: JADX, Bytecode Viewer.
Dynamic: Frida (spawn or attach), adb run-as, Android Studio debugger.
If “run-as permission denied”:
Ensure this is a debug build or that the app is marked debuggable.
Otherwise use Frida to read SharedPreferences via Java APIs at runtime.

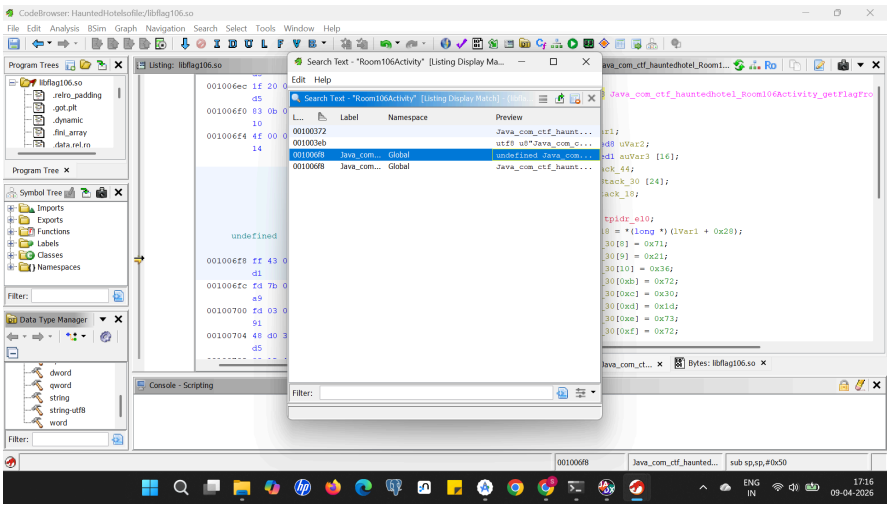


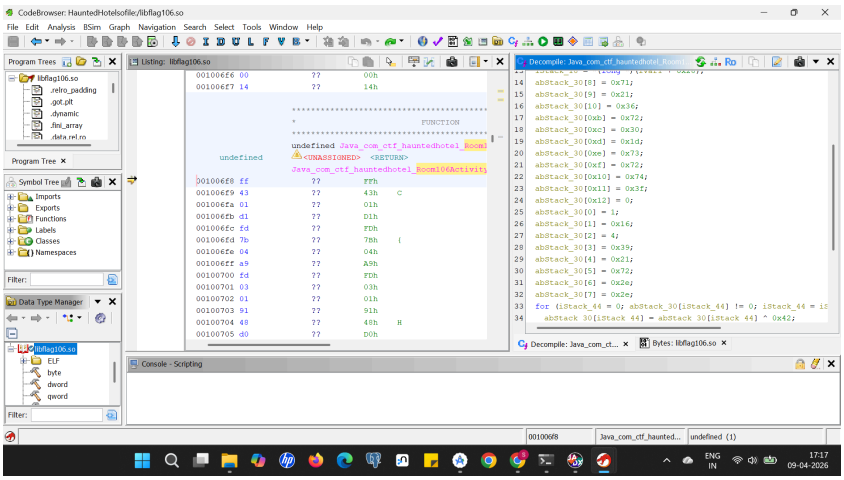


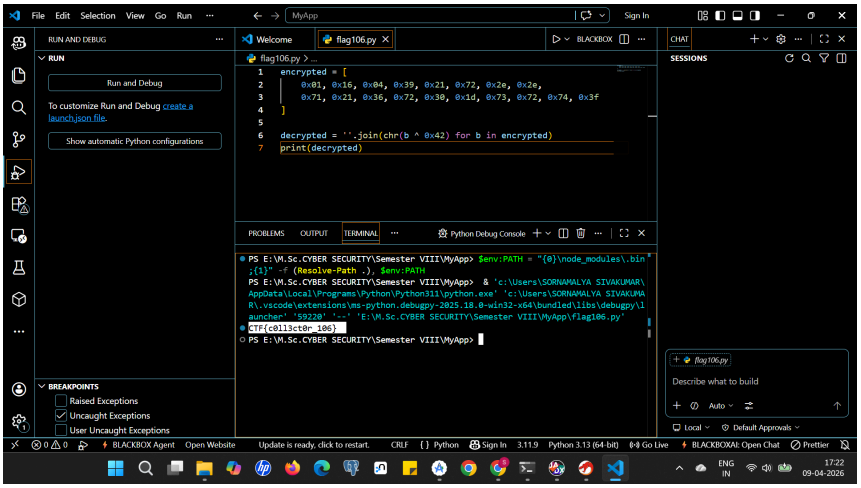
FLAG 6 - In Native File

1.Decompile the APK using jadx and locate Room106Activity loading libflag106.so.
2.Extract the native library from the APK and open it in Ghidra or IDA Pro.
3.Find the XOR decryption loop with key 0x42 and the encrypted byte array.
4.XOR decrypt the bytes to obtain CTF{c0ll3ct0r_106}


Windows PowerShell
+ ~~~~~~~
+ CategoryInfo : ObjectNotFound: (unzip:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
PS E:\M.Sc.CYBER SECURITY\Semester VIII\MyApp\unZipDir> tar -xf HauntedHotel.apk
PS E:\M.Sc.CYBER SECURITY\Semester VIII\MyApp\unZipDir> ls
Directory: E:\M.Sc.CYBER SECURITY\Semester VIII\MyApp\unZipDir
Mode LastWriteTime Length Name
d----- 09-04-2026 16:34 assets
d----- 09-04-2026 16:34 kotlin
d----- 09-04-2026 16:34 lib
d----- 09-04-2026 16:34 META-INF
d----- 09-04-2026 16:34 res
-a---- 01-01-1981 01:01 7540 AndroidManifest.xml
-a---- 01-01-1981 01:01 11091984 classes.dex
-a---- 01-01-1981 01:01 481820 classes2.dex
-a---- 01-01-1981 01:01 78936 classes3.dex
-a---- 01-01-1981 01:01 3080 classes4.dex
-a---- 01-01-1981 01:01 745624 classes5.dex
-a---- 01-01-1981 01:01 1728 DebugProbesKt.bin
-a---- 08-04-2026 07:20 8837854 HauntedHotel.apk
-a---- 01-01-1981 01:01 1150232 resources.arsc
PS E:\M.Sc.CYBER SECURITY\Semester VIII\MyApp\unZipDir>

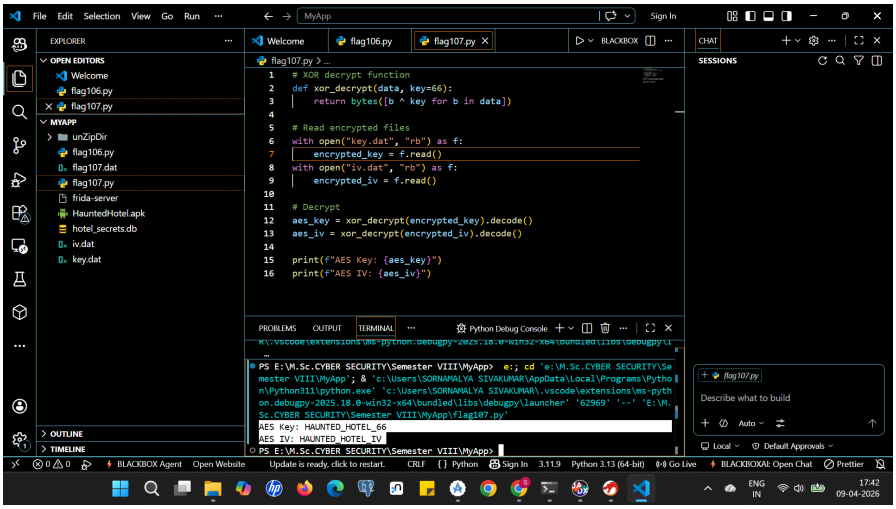

CodeBrowser: HauntedHotelsofile/libflag106.so
Search Text - "Room106Activity" [Listing Display Ma...
Search Text - "Room106Activity" [Listing Display Match]
Java_com_ctf_hauntedhotel_Room106Activity_getFlagFro

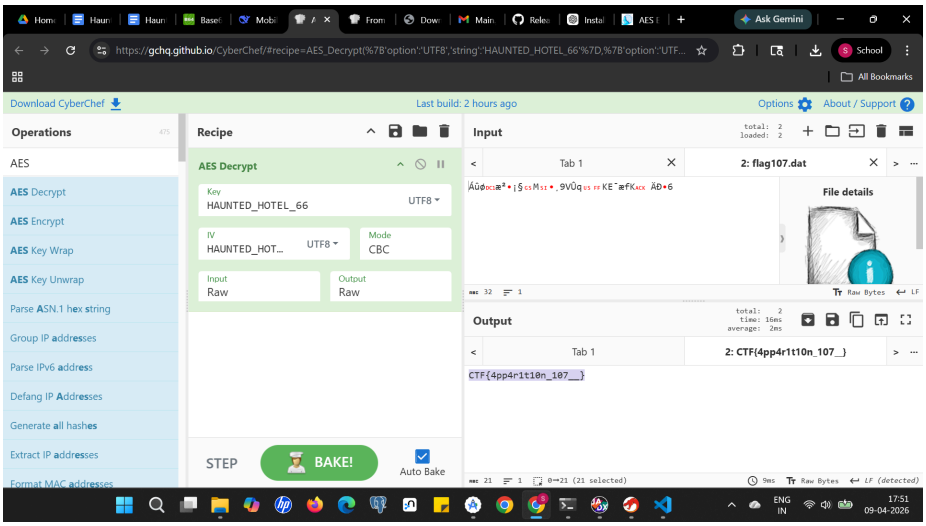

CodeBrowser: HauntedHotelsofile/libflag106.so
Listing: libflag106.so
Decompile: Java_com_ctf_hauntedhotel_Room
FUNCTION
undefined Java_com_ctf_hauntedhotel_Room
<UNASSIGNED> <RETURN>
Java_com_ctf_hauntedhotel_Room106Activity
abStack_30[8] = 0x71;
abStack_30[9] = 0x21;
abStack_30[10] = 0x36;
abStack_30[0xb] = 0x72;
abStack_30[0xc] = 0x30;
abStack_30[0xd] = 0x1d;
abStack_30[0xe] = 0x73;
abStack_30[0xf] = 0x72;
abStack_30[0x10] = 0x74;
abStack_30[0x11] = 0x3f;
abStack_30[0x12] = 0;
abStack_30[0] = 1;
abStack_30[1] = 0x16;
abStack_30[2] = 4;
abStack_30[3] = 0x39;
abStack_30[4] = 0x21;
abStack_30[5] = 0x72;
abStack_30[6] = 0x2e;
abStack_30[7] = 0x2e;
for (iStack_44 = 0; abStack_30[iStack_44] != 0; iStack_44 = is
abStack_30[iStack_44] = abStack_30[iStack_44] ^ 0x42;

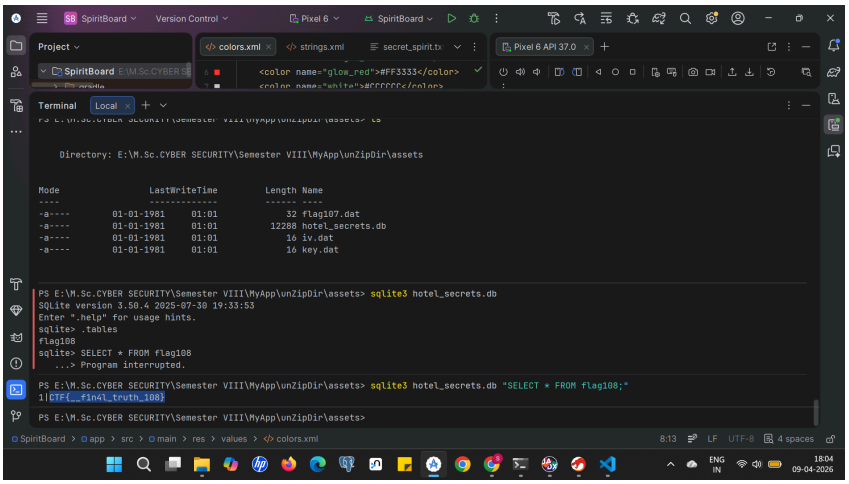


FLAG 7 - Asset file + AES encryption

1.Decompile the APK using jadx and locate Room107Activity reading flag107.dat from assets.
2.Extract flag107.dat from the APK assets folder using apktool or by unzipping.
3.Find the key and IV .bat files from the asset and read the binary to find the key and IV of AES .
4.Decrypt flag107.dat using AES with the found key and IV to get the secret CTF{4pp4r1t10n_107__} in CyberChef.





FLAG 8 - In sqlite database

1.Decompile the APK using jadx and locate FlagDatabaseHelper.java copying hotel_secrets.db from assets.
2.Extract hotel_secrets.db from the APK assets folder using apktool or by unzipping.
3.Open the database and list all the tables.
4.Query the flag108 table to retrieve CTF{__f1n4l_truth_108}